

Before starting, make sure you **read all the questions** and understand what is asked of you. Write your answers to the questions in a separate sheet of paper and make sure you **justify yes/no answers**. Multiple-choice questions have a **unique correct answer**.

1. (20 points) Translate the following C snippet into LLVM intermediate representation and into MIPS.

```
int ackermann(int x, int y) {
    if (x < 0 || y < 0)
        return -1;
    if (x == 0)
        return y + 1;
    if (y == 0)
        return ackermann(x - 1, 1);
    return ackermann(x - 1, ackermann(x, y - 1));
}
```

2. (5 points) Suppose we are given a *deterministic finite automaton* for a language $\mathcal{L}$ on Σ . For all words $w \in \Sigma^*$, we can use the automaton to determine whether $w \in \mathcal{L}$ in time:

- A. $\mathcal{O}(2^{|w|})$
- B. $\mathcal{O}(|w|^2)$
- C. $\mathcal{O}(|w|)$
- D. $\mathcal{O}(\log |w|)$

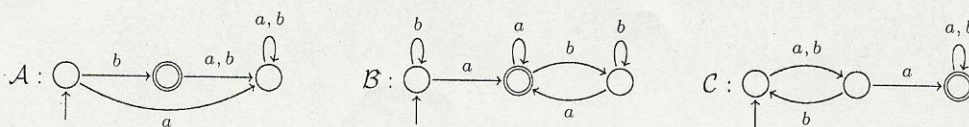
3. (5 points) Suppose we are given a *context free grammar* for a language $\mathcal{L}$ on Σ . For all words $w \in \Sigma^*$, we can use the grammar to determine whether $w \in \mathcal{L}$ in time:

- A. polynomial in $|w|$
- B. exponential in $|w|$
- C. nope, that is an undecidable problem

4. (6 points) Split the following words into tokens (that is, lexeme-word pairs)

- (a) (4 points) *baaaab*
- (b) (4 points) *bbba*
- (c) (4 points) *abbbab*

using the following automata with **decreasing priority from left to right**. Tip: remember to follow the “maximal munch” principle.



5. (20 points) Let $T = \{\text{id}, +\}$ be a set of terminals. Is the following grammar LL(1)? Does it define an LL(2) language?

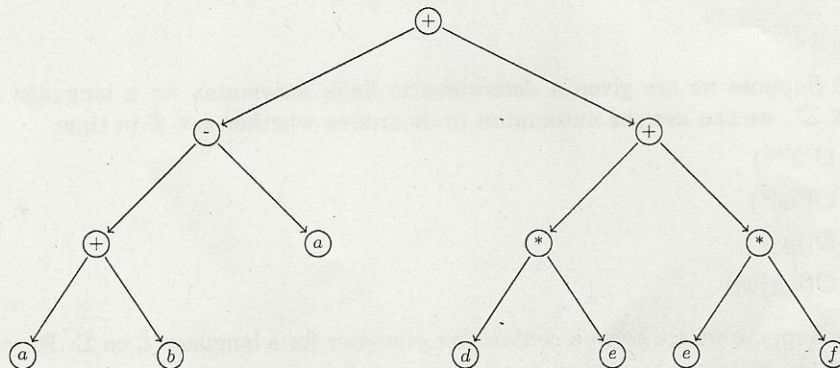
(1)	$S \rightarrow E$
(2)	$E \rightarrow \text{id} + C$
(3)	$C \rightarrow \text{id}$

6. (14 points) During the course we have focused on conservative approximations of both type checking and reachability analysis. Argue that the exact versions of both tasks are in fact *undecidable*.

7. (20 points) Let $T = \{\text{cst}, \text{id}, +, *\}$ be a set of terminals. Is the following grammar SLR(0)? Is it LALR(2)?

(1)	Exp	$\rightarrow$	Exp + Prod
(2)		$\rightarrow$	Prod
(3)	Prod	$\rightarrow$	Prod * atom
(4)		$\rightarrow$	atom
(5)	atom	$\rightarrow$	cst
(6)		$\rightarrow$	id

8. (5 points) What is the minimal number of registers required to evaluate the expression tree shown below without using algebraic properties, nor additional memory?



9. (5 points) In the above question, can you do better if you use algebraic identities? Why or why not?